Index

1 Introduction **1**

1.1 Motivation 1

1.2 Problem statement 2

1.3 Objectives 2

2 Implementation **3**

2.1 Architecture and technology 3

2.2 Database design 4

2.3 Pseudocode 5

2.4 Key code excerpts 11

3 Results **13**

3.1 Running the system 13

3.2 Output screens 14

3.3 Functional testing 17

3.4 Security checks 17

3.5 Problems found and fixed 17

4 Conclusion **17**

4.1 How the objectives were met 20

4.2 What we learned 21

4.3 Future scope 21

1 Introduction

Every campus has the same small problem. Somebody leaves a calculator in the library, somebody else picks it up and has no idea whose it is, and the two of them never meet. Phones, identity cards, keys, bottles and wallets go missing all the time, and most of them could be returned if the right two people could find each other.

The Campus Lost & Found Portal is a web application built to do exactly that. A student who loses something posts a report, a student who finds something posts one too, and anyone can search the reports. We built it for the subject Advanced Java and J2EE. It has a browser front end made with JSP, HTML and CSS, a back end made with Servlets, filters and listeners running on Apache Tomcat, and a relational database reached through JDBC. The code is organised with the Model View Controller (MVC) pattern and Data Access Objects (DAO), which is the usual way of structuring Java web applications.

1.1 Motivation

On our campus, a lost object is usually handled by luck. People put a note on a notice board, send a message to a class group, or walk to the security desk and hope. All of these work a little, and all of them have the same weaknesses.

The information is scattered. A message in one group is never seen by the students who are not in it, and nobody can look in one place to see everything that has been lost or found. There is also no search. Group messages scroll away within hours, so checking whether a black wallet was handed in last Tuesday means scrolling through hundreds of messages by hand.

Trust is the bigger problem. A finder who posts that a calculator has been found cannot tell whether the person who replies is the real owner, because anybody can say anything. Many people also post their phone number openly on notice boards and in groups, where strangers can copy it. Finally, nobody removes a notice once the item is back with its owner, so people keep spending time on cases that are already closed.

The subject gave us a second reason. Advanced Java is easiest to understand when many of its topics are used together: servlets handling requests, JSP pages building responses, sessions remembering users, filters guarding pages, listeners reacting to events, and JDBC with transactions keeping data correct. A lost and found system needs all of these, and it is a problem every student already understands, so we could spend our time on the Java instead of on the idea.

1.2 Problem statement

To design and build a secure, database backed web portal where campus members can register and log in, post reports of lost and found objects, search all the reports, and contact each other about an object without their personal contact details being shown until the owner of the report has checked the request and approved it.

To make this concrete, the system has to answer five practical questions. How does a person describe a lost or found object well enough for others to recognise it? How can another person find a matching report quickly among many? How can the owner of a found object check that a claimant is genuine, without having to trust a stranger? How are phone numbers and e-mail addresses kept private and shown only to the right person at the right time? And how does the application keep its data correct when something fails half way through an operation that changes several records?

1.3 Objectives

The aims of the project were the following.

1. Build a working web application with a front end and a back end, using Servlets, JSP, JSTL and JDBC and no large frameworks, so that every part can be explained.
2. Let users register and log in, with checks on the server, salted password hashes and session based login.
3. Let a logged in user post a Lost or Found report with a title, description, category, place and date.
4. Provide search and filters by keyword, type, category and status.
5. Provide a claim process: a user sends a message about someone else's item, and the owner approves or rejects it.
6. Show the owner's e-mail and phone number only to a claimant whose claim has been approved.
7. Allow only the owner of an item to resolve it, delete it or decide on claims about it, and require a login for every page except login and sign-up.
8. Use JDBC transactions for every operation that changes more than one row.
9. Use filters for login protection, character encoding and caching, and listeners for start-up work and session counting.

10. Make the project easy to run, so that one command starts the server and the database on any computer with Java.

2 Implementation

2.1 Architecture and technology

The application follows MVC with the DAO pattern, and every request takes the same path, shown in Figure 1. The browser sends a request. The filters run first. A servlet, which acts as the controller, reads the input and asks a DAO to read or write the database. The servlet then puts the result in the request and forwards it to a JSP, the view, which produces the HTML that goes back to the browser.

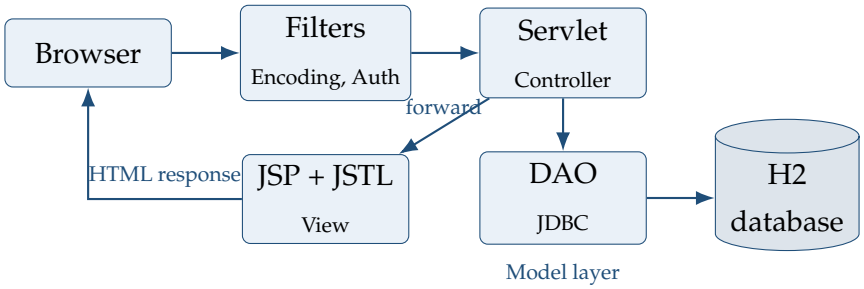


Figure 1: How a request moves through the application.

Table 1 lists what we used. The server is Tomcat, but it is started from a normal `main` method inside the project (embedded Tomcat), so nobody needs to install or configure a server. The database is H2, which also runs inside the program and stores its files in the user's home folder. We run H2 in its MySQL mode and write plain SQL, so moving to MySQL only needs a change in one properties file.

Table 1: Technology used.

Part	Technology
Language and platform	Java 17 or later (built and tested on JDK 21), Jakarta EE 10, Servlet 6.0, JSP 3.1
Web server	Apache Tomcat 10.1, started as an embedded server
View	JSP with EL and JSTL (no Java code inside the pages), HTML5 and CSS3
Database access	JDBC with `PreparedStatement`, on the H2 embedded database in MySQL mode
Build tool	Apache Maven; `mvn compile exec:java` starts everything
Security helpers	`MessageDigest` (SHA-256), `SecureRandom`, `HttpSession`

The source has about 1700 lines of Java and 700 lines of JSP and CSS. It is split into packages that follow MVC, and Table 2 lists the main classes and what each one does.

Table 2: Main classes of the application.

Class	URL	What it does
`HomeServlet`	`/`	Home page with live counts and the latest reports
`RegisterServlet`	`/register`	Shows and processes the sign-up form
`LoginServlet`	`/login`	Checks the credentials and starts the login session
`LogoutServlet`	`/logout`	Ends the session (POST only)
`ItemListServlet`	`/items`	Browse, search and filter reports
`PostItemServlet`	`/post`	Shows and processes the new report form
`ItemDetailServlet`	`/item`	One report; what is shown depends on who is looking
`ClaimServlet`	`/claim`	Sends a claim message about an item
`ClaimDecisionServlet`	`/decide`	Owner approves or rejects a claim
`ManageItemServlet`	`/manage`	Owner resolves or deletes an item
`MyItemsServlet`	`/my-items`	The user's own reports and sent claims
`AuthFilter`	`protected URLs`	Redirects to login when nobody is logged in
`EncodingFilter`	`all URLs`	UTF-8 encoding and no-cache headers
`AppStartupListener`	`(context)`	Creates the tables and demo data at start
`SessionCounterListener`	`(sessions)`	Counts the live sessions
`UserDAO, ItemDAO, ClaimDAO`	`DAO layer`	All the SQL for the three tables
`PasswordUtil, WebUtil, DBUtil`	`helpers`	Hashing, request helpers, connections

2.2 Database design

The data is kept in three tables (Figure 2). A user can post many items, an item can receive many claims, and every claim is made by one user. Items carry a status of OPEN or RESOLVED, and claims carry PENDING, APPROVED or REJECTED, so the progress of every case is always recorded. Foreign keys stop an item or a claim from pointing at something that does not exist.

Two decisions in this design are worth explaining. First, the password itself is never stored. We keep a random salt and the hash of the salt joined with the password, so even someone who reads the database cannot get the passwords back. Second, a claim does not carry the owner's contact details. They stay in the users table and are fetched

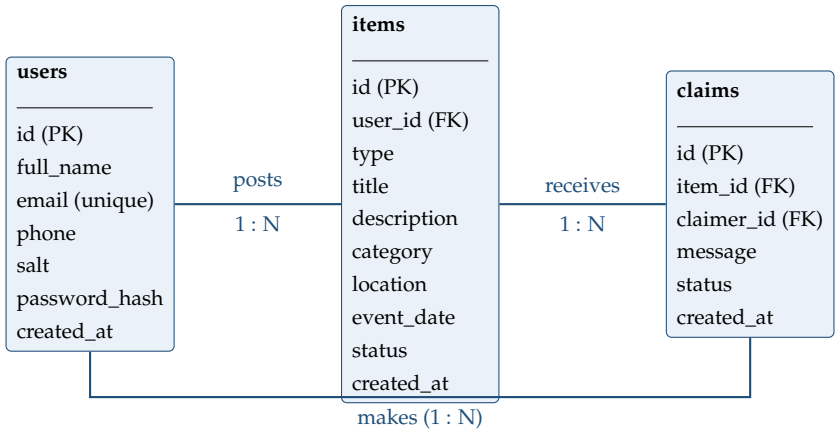


Figure 2: Entity relationship diagram.

with a join only when the program has decided that the person looking is allowed to see them. This keeps the privacy rule in one place.

Figure 3 shows how items and claims change state. Approving a claim is the only thing that resolves an item automatically, and doing so also rejects every other pending claim on that item.

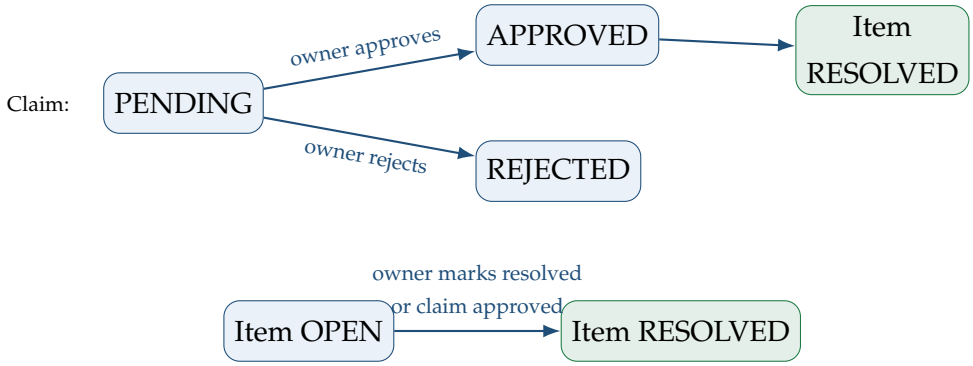


Figure 3: State changes of a claim and of an item.

2.3 Pseudocode

This section describes the main operations in pseudocode. Names such as FINDBYEMAIL are DAO methods, and each of them runs one SQL statement with placeholders. FORWARD means passing the request to a JSP, and REDIRECT means telling the browser to load another address. After a successful form submission we always redirect, so that refreshing the page does not send the form again. This is known as the Post Redirect Get pattern.

2.3.1 Start-up

When the server starts, a listener prepares everything the application needs (Algorithm 1).

Algorithm 1 Application start-up (AppStartupListener)

```
open a database connection
for each statement in schema.sql: execute it        ▷ CREATE TABLE IF NOT EXISTS
if table users is empty then
    create demo users (salted hash of the demo password)
    insert sample Lost and Found items and one sample claim
store a counter activeSessions ← 0 in the application scope
store the category list in the application scope
```

2.3.2 Registration and password hashing

The server repeats every check the browser makes, because the browser cannot be trusted. Passwords are saved as a salted SHA-256 hash (Algorithms 2 and 3).

Algorithm 2 Register a new user (RegisterServlet, POST)

Input: name, e-mail, phone, password, confirmation

```
errors ← empty list; email ← lower-case(trim(email))
if length(name) ∉ [2, 100] then add error
if email does not match x@y.z pattern then add error
if phone is given and not 7 to 15 digits, plus, minus or space then add error
if length(password) < 6 or password ≠ confirmation then add error
if errors is empty and FINDBYEMAIL(email) exists then add error "already registered"
if errors is not empty then
    FORWARD(register page with errors and the typed values); return
salt ← 16 random bytes in hex; hash ← HASH(password, salt)
INSERT(user with salt and hash)
flash message "Account created"; REDIRECT(login)
```

Algorithm 3 Salted hashing and verification (PasswordUtil)

```
function HASH(password, salt)
    return hex(SHA-256(salt + password))
function VERIFY(password, salt, storedHash)
    return HASH(password, salt) = storedHash
```

2.3.3 Login, logout and the login filter

A user logs in with an e-mail and a password. The servlet never compares passwords directly. It finds the user by e-mail, hashes the typed password with that user's salt, and compares the result with the stored hash (Algorithm 4). The filter in Algorithm 5 is what keeps every other page closed to people who are not logged in.

Algorithm 4 Login (LoginServlet, POST)

Input: e-mail, password

1 *user* ← FINDBYEMAIL(lower-case(email))
2 **if** *user* is null **or not** VERIFY(password, user.salt, user.hash) **then**
3 FORWARD(login page, "Invalid e-mail or password") ▷ same message for both cases
4 **return**
5 remove salt and hash from *user* ▷ never keep them in the session
6 *target* ← session attribute `afterLogin`
7 change the session id ▷ prevents session fixation
8 session.`user` ← *user*; set inactivity time-out to 30 minutes
9 flash message "Welcome, name"
10 **if** *target* starts with "/" **and not** "//" **then** REDIRECT(*target*)
11 **else** REDIRECT(home)

Algorithm 5 Authentication filter (AuthFilter)

Input: request for one of: `/items /item /post /my-items /claim /manage /decide`

1 *session* ← existing session or null (do not create one)
2 **if** *session* ≠ null **and** session.`user` ≠ null **then**
3 pass the request on to the servlet; **return**
4 **if** method is GET **then**
5 session.`afterLogin` ← path and query string requested
6 flash message "Please log in to continue"
7 REDIRECT(login)

The home page servlet makes the same check, so opening the site always leads a visitor who is not logged in to the login page. Logout is a POST request that invalidates the session, which removes the user and everything else stored in it. A session listener keeps count of the live sessions (Algorithm 6).

Algorithm 6 Logout and session counting

1 **LogoutServlet (POST):** session.INVALIDATE(); flash "You have been logged out"; REDIRECT(home)
2 **sessionCreated:** `activeSessions` ← `activeSessions` $+1$ ▷ atomic
3 **sessionDestroyed:** `activeSessions` ← `activeSessions` -1

2.3.4 Posting and searching

Algorithm 7 Post a Lost or Found report (PostItemServlet, POST)

Input: type, title, description, category, location, date

validate: type $\in$ {LOST, FOUND}; title 3 to 120 characters; description at most 1000 characters
validate: category is in the allowed list; location 1 to 120 characters; date is valid and **not in the future**
if any error **then** FORWARD(form with messages and typed values); **return**
item $\leftarrow$ new Item(owner = current user, status = OPEN, ...)
id $\leftarrow$ INSERT(*item*) ▷ PreparedStatement
flash "Your report has been posted"; REDIRECT(`/item?id=`*id*)

The search builds its SQL step by step, but the values always go in through `?` placeholders and are never pasted into the SQL text. This is what protects the application from SQL injection (Algorithm 8). The status filter is OPEN by default, and the value ALL removes it, so resolved reports can still be looked at.

Algorithm 8 Search and filter (ItemDAO.search)

Input: type, category, keyword, status (each may be empty)

sql $\leftarrow$ "SELECT ... FROM items JOIN users ... WHERE 1 = 1"; *params* $\leftarrow$ empty list
if type given **then** append "AND type = ?"; add type to *params*
if category given **then** append "AND category = ?"; add category to *params*
if status given **then** append "AND status = ?"; add status to *params*
if keyword given **then**
    append "AND (LOWER(title) LIKE ? OR LOWER(description) LIKE ? OR LOWER(location) LIKE ?)"
    add "%keyword%" (lower-case) to *params* three times
append "ORDER BY created_at DESC"
prepare *sql*; bind every value in *params* in order; execute
return list of Items built from the result rows

2.3.5 Viewing an item and sending a claim

The item page shows different things to different people (Algorithm 9). This is also where the contact details are protected.

Algorithm 9 Item detail page (ItemDetailServlet, GET)

```
item ← FINDBYID(id); if null then send HTTP 404; return
isOwner ← (current user id = item.ownerId)
if isOwner then
    request.claims ← all claims on the item                ▷ with Approve / Reject buttons
else
    my ← FINDBYITEMANDCLAIMER(id, current user)
    request.showOwnerContact ← (my ≠ null and my.status = APPROVED)
FORWARD(item-detail page)
```

Algorithm 10 Send a claim message (ClaimServlet, POST)

Input: itemId, message

```
item ← FINDBYID(itemId); if null then HTTP 404
if item belongs to current user then flash "You cannot claim your own item"
else if item.status ≠ OPEN then flash "This item is already resolved"
else if length(message) ∉ [5, 500] then flash "Message must be 5 to 500 characters"
else if FINDBYITEMANDCLAIMER(itemId, user) exists then flash "Already sent"
else
    INSERTCLAIM(itemId, user, message) with status PENDING
    flash "Your message was sent to the owner"
REDIRECT(/item?id=itemId)
```

2.3.6 Deciding on a claim

Approving a claim changes three things: the claim itself, the other pending claims on the item, and the status of the item. If the program stopped after the first change, the item would stay open even though a claim had been approved. To prevent that, the three updates run as one transaction, so either all of them are saved or none are (Algorithm 11).

As an example, suppose three students have sent a message about the same lost calculator, and the owner approves the second one. In a single transaction the program then does three things:

1. the second claim becomes APPROVED;
2. the first and third claims become REJECTED;
3. the calculator's report becomes RESOLVED.

If the database failed after the second step, the rollback would restore all three claims to PENDING and leave the item open, so the owner could simply try again. Without a transaction the item could end up with an approved claim and still be listed as open.

Algorithm 11 Owner decides on a claim (ClaimDecisionServlet and ClaimDAO.decide)

Input: claimId, decision ∈ {approve, reject}

```
1  claim ← FINDBYID(claimId); item ← FINDBYID(claim.itemId)
2  if claim null then HTTP 404
3  else if current user ≠ item.owner then HTTP 403 "Only the owner"
4  if claim.status ≠ PENDING or item.status ≠ OPEN then flash "Already dealt with"; REDIRECT; return
5  connection.SETAUTOCOMMIT(false)                                   ▷ begin transaction
6  try:
7     UPDATE claims SET status = (APPROVED or REJECTED) WHERE id = claimId
8  if decision = approve then
9        UPDATE claims SET status = REJECTED WHERE item = itemId AND id ≠ claimId AND status = PENDING
10       UPDATE items SET status = RESOLVED WHERE id = itemId
11    connection.COMMIT()
12 on SQL error: connection.ROLLBACK(); report the error
13 REDIRECT(/item?id=item.id)
```

2.3.7 Resolving and deleting an item

Both actions are only for the owner. The check is repeated on the server even though other users never see the buttons, because hiding a button does not protect anything (Algorithm 12).

Algorithm 12 Manage an item (ManageItemServlet, POST)

Input: itemId, action ∈ {resolve, delete}

```
1  item ← FINDBYID(itemId); if null then HTTP 404
2  if item.owner ≠ current user then HTTP 403; return
3  if action = resolve then UPDATE items SET status = RESOLVED; REDIRECT(item page)
4  else if action = delete then
5     begin transaction
6     DELETE FROM claims WHERE item = itemId          ▷ child rows first (foreign key)
7     DELETE FROM items WHERE id = itemId
8     commit (rollback on error); flash "Item deleted"; REDIRECT(my items)
9  else HTTP 400
```

Deleting an item needs a transaction as well. The claims about the item are deleted first, because a claim cannot exist without its item (the foreign key would refuse it). If the second delete failed, the first would be undone and nothing would change.

2.3.8 The common filter

Algorithm 13 EncodingFilter (every request)

```
set request and response character encoding to UTF-8
if the path is not a stylesheet then
    set header Cache-Control: no-store, no-cache, must-revalidate
pass the request on to the next filter or servlet
```

The no-cache header means that after logging out, the Back button of the browser cannot bring back a private page from its cache.

2.4 Key code excerpts

Pseudocode shows the idea, so here are four short pieces of the real source code as well. They are copied from the project without changes.

The first is the login filter. It lets a logged in user through, and otherwise remembers the page that was asked for and sends the visitor to the login page.

```
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
    FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);           // false = do not
    create a new session
        boolean loggedIn = session != null && session.getAttribute("user") != null;

        if (loggedIn) {
            chain.doFilter(request, response);                  // allowed -
    continue to the servlet
            return;
        }

        // Not logged in: remember where he/she wanted to go (only for simple GET
    page requests)
        if ("GET".equals(req.getMethod())) {
            // full path inside the application (servletPath alone would drop any
    extra path and so lose where he/she wanted to go)
            String target =
    req.getRequestURI().substring(req.getContextPath().length());
            if (req.getQueryString() != null) {
                target += "?" + req.getQueryString();
            }
            req.getSession().setAttribute("afterLogin", target);
        }
        WebUtil.flash(req, "info", "Please log in to continue.");
        resp.sendRedirect(req.getContextPath() + "/login");
    }
```

Passwords are hashed with SHA-256 over the salt and the password. To check a login we hash the typed password with the stored salt and compare the two results.

```
    public static String hash(String password, String salt) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] digest = md.digest((salt +
    password).getBytes(StandardCharsets.UTF_8));
            return toHex(digest);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 not available", e);
        }
    }

    public static boolean verify(String password, String salt, String
    expectedHash) {
        return hash(password, salt).equals(expectedHash);
    }
```

The search method adds one condition to the SQL for every filter that was filled in and keeps the values in a list, which is bound to the ? marks later.

```
    public List<Item> search(String type, String category, String keyword, String
    status, int limit)
            throws SQLException {
        StringBuilder sql = new StringBuilder(BASE_SELECT).append("WHERE 1 = 1");
        List<Object> params = new ArrayList<>();

        if (notBlank(type)) {
            sql.append(" AND i.type = ?");
            params.add(type);
        }
        if (notBlank(category)) {
            sql.append(" AND i.category = ?");
            params.add(category);
        }
        if (notBlank(status)) {
            sql.append(" AND i.status = ?");
            params.add(status);
        }
        if (notBlank(keyword)) {
            sql.append(" AND (LOWER(i.title) LIKE ? OR LOWER(i.description) LIKE ?
    OR LOWER(i.location) LIKE ?)");
            String like = "%" + keyword.toLowerCase() + "%";
            params.add(like);
            params.add(like);
            params.add(like);
        }
        sql.append(" ORDER BY i.created_at DESC, i.id DESC");
        if (limit > 0) {
            sql.append(" LIMIT ").append(limit);
        }
```

Finally, the transaction used when the owner decides on a claim. Auto commit is switched off, the updates are made, and the changes are saved with `commit()`. If any update fails, `rollback()` cancels all of them.

```
    public void decide(int claimId, int itemId, boolean approve) throws
    SQLException {
        try (Connection con = DBUtil.getConnection()) {
            con.setAutoCommit(false);
            try {
                try (PreparedStatement ps = con.prepareStatement("UPDATE claims
    SET status = ? WHERE id = ?")) {
                    ps.setString(1, approve ? "APPROVED" : "REJECTED");
                    ps.setInt(2, claimId);
                    ps.executeUpdate();
                }
                if (approve) {
                    try (PreparedStatement ps = con.prepareStatement(
                            "UPDATE claims SET status = 'REJECTED' WHERE item_id =
    ? AND id <> ? AND status = 'PENDING'")) {
                        ps.setInt(1, itemId);
                        ps.setInt(2, claimId);
                        ps.executeUpdate();
                    }
                    try (PreparedStatement ps = con.prepareStatement("UPDATE items
    SET status = 'RESOLVED' WHERE id = ?")) {
                        ps.setInt(1, itemId);
                        ps.executeUpdate();
                    }
                }
                con.commit();
            } catch (SQLException e) {
                con.rollback();
                throw e;
            }
        }
    }
```

3 Results

3.1 Running the system

The whole system, including the web server and the database, starts with one command, `mvn compile exec:java`. The first time, Maven downloads the libraries, which takes a few minutes. After that, start-up takes only a few seconds. When the application is ready the console prints that the database is ready and shows the address `http://localhost:8080/`. Opening that address always leads to the login page for a visitor who has not logged in. Two demo accounts and six sample reports are created automatically, so the whole process can be shown straight away.

3.2 Output screens

The following screens were taken from the running application. Figure 4 is the login page, which is the first thing anyone sees, and Figure 5 is the sign-up form. After logging in, the Home page (Figure 6) shows how many lost and found reports are open and how many items have been reunited, followed by the newest reports. The Browse page (Figure 7) has the search box and the filters for type, category and status. Figure 8 is the form used to post a report. Figure 9 is the My items page, where a user sees the reports they posted and the messages they have sent, and Figure 10 is the page shown when an address does not exist.

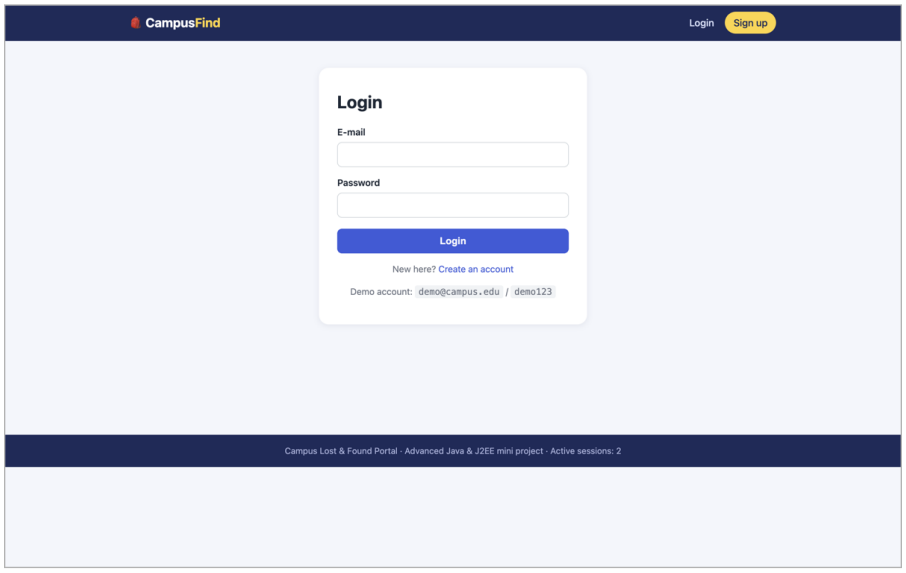


Figure 4: Login page.

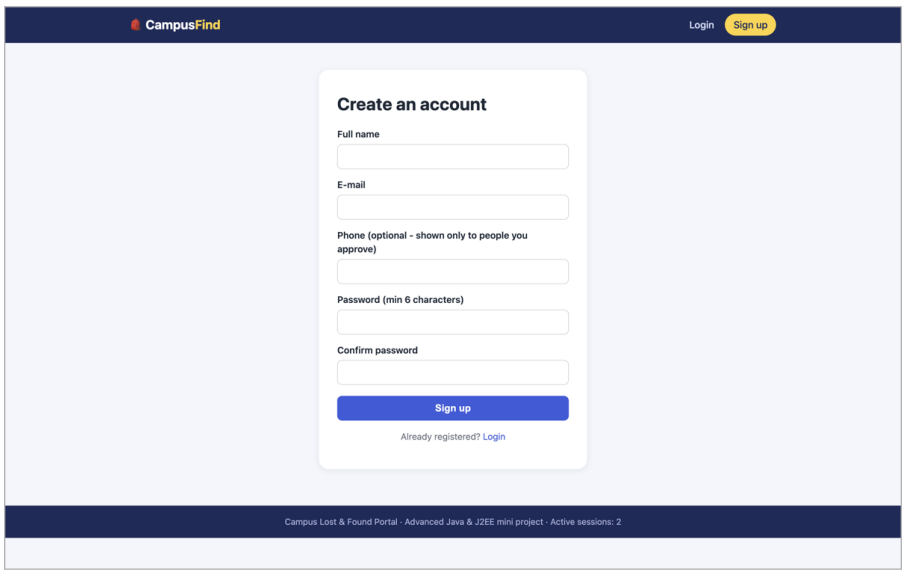


Figure 5: Sign-up page.

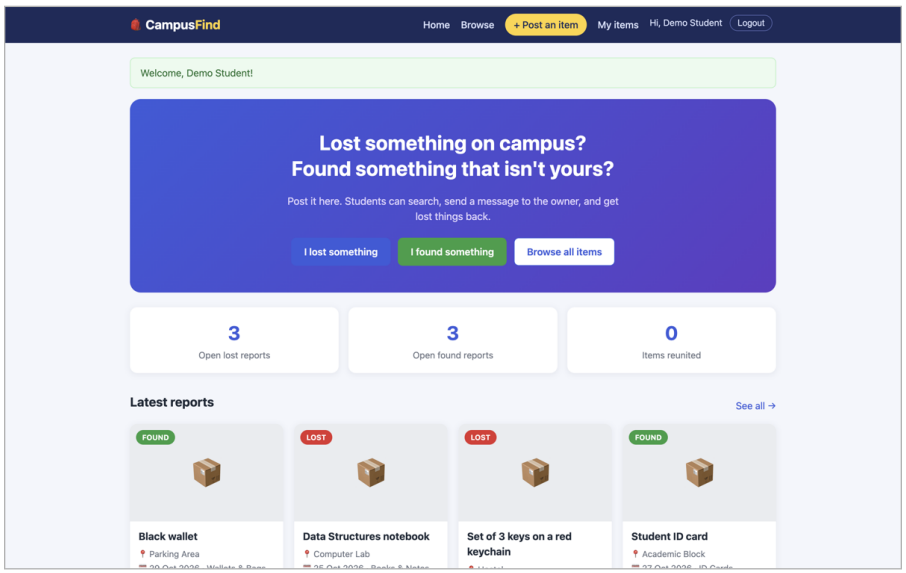


Figure 6: Home page with the live counts and latest reports.

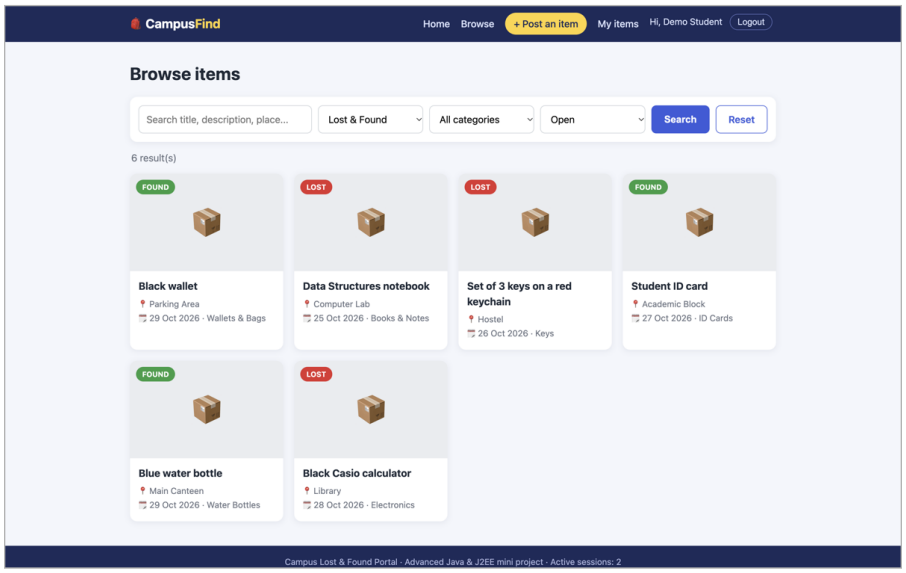


Figure 7: Browse page with the search box and filters.

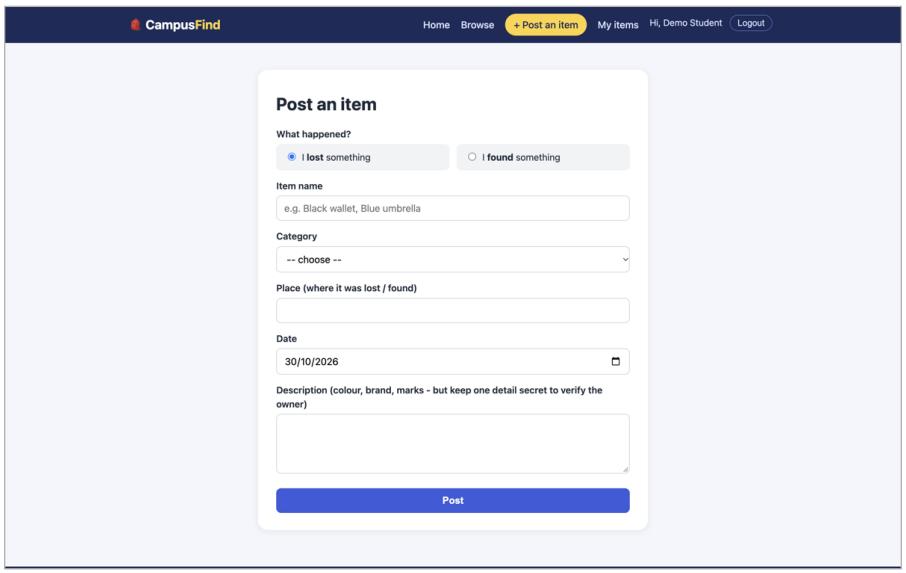


Figure 8: Form for posting a Lost or Found report.

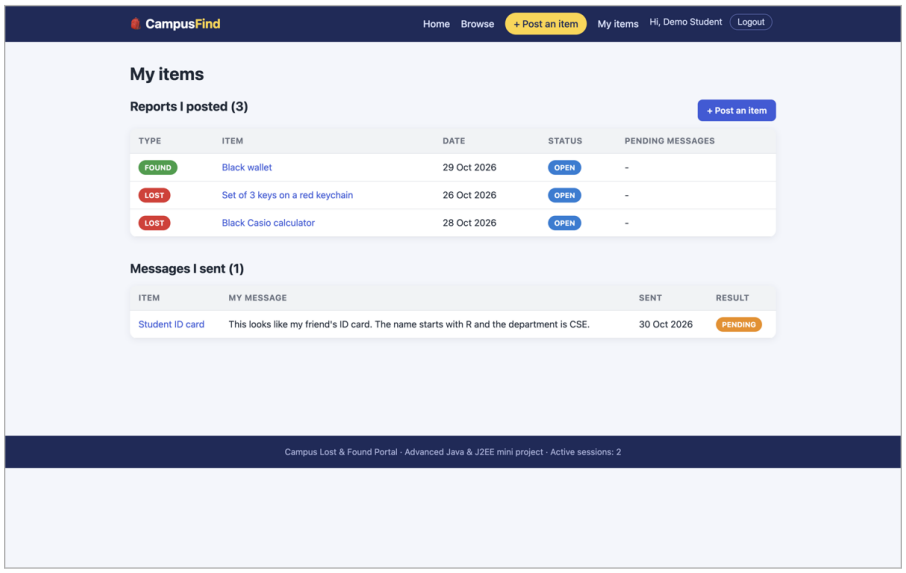


Figure 9: My items page: reports posted and messages sent.

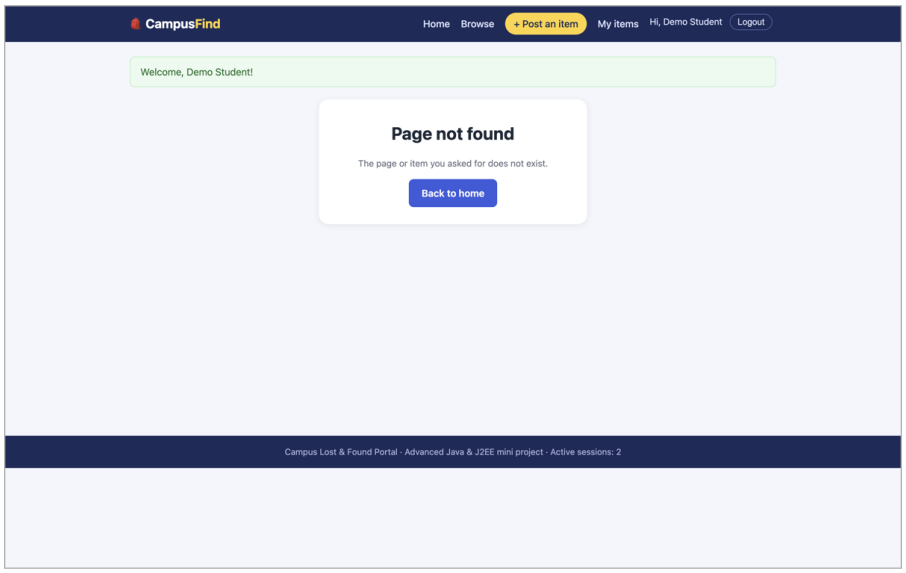


Figure 10: Friendly page shown for an address that does not exist.

3.3 Functional testing

Every feature was tried in a real browser using several test accounts: an owner, a claimant and the two demo accounts. Table 3 lists the main cases and what happened.

3.4 Security checks

Table 4 lists the threats we thought about, what the application does about each, and whether we tried it in the browser or only checked it in the code.

3.5 Problems found and fixed

Testing in the browser also showed several faults, which we fixed before the final version. They are listed in Table 5. Most of them would never have been noticed by only reading the code.

4 Conclusion

The Campus Lost & Found Portal turns an informal and unreliable process into an organised one. A person who loses or finds something can post a report in under a minute, everyone can search all reports in one place, and contact details are shared only after the owner has read the claimant's message and approved it. A stranger can no longer get a phone number or e-mail address just by asking. Every test case in Table 3 passes, and the application runs on a normal laptop from a single command.

Table 3: Test cases and results.

No.	Test	Expected behaviour	Result
1	Open the site while logged out	Redirected to the login page	Pass
2	Open Browse or an item URL while logged out	Redirected to login with a message; after login the requested page opens	Pass
3	Register with a short password	Rejected with a message	Pass
4	Register with different confirmation	“Passwords do not match”; typed values kept	Pass
5	Register with an e-mail that exists	Rejected as a duplicate	Pass
6	Register with valid data	Account created, redirected to login	Pass
7	Login with a wrong password	“Invalid e-mail or password”	Pass
8	Login with correct data	Welcome message, session created	Pass
9	Home page counts	Open Lost, open Found and reunited counts change after posting or resolving	Pass
10	Keyword search, for example “wallet”	Only matching reports shown	Pass
11	Filter by type, category and status, alone and together	Correct reports; empty message when nothing matches	Pass
12	Post with empty fields	Messages from the server, nothing saved	Pass
13	Post a valid Lost and a valid Found report	Saved; the new item page opens	Pass
14	Owner opens own item	Sees Resolve, Delete and the claims	Pass
15	Another user opens the item	Sees the claim form, no owner contact	Pass
16	Send an empty or very short message	Rejected with a message	Pass
17	Send a valid claim	Status PENDING; the form is replaced by the sent message (no duplicates)	Pass
18	Owner dashboard	Shows “1 waiting” for the pending claim	Pass
19	Owner approves the claim	Item becomes RESOLVED; claimant now sees owner e-mail and phone	Pass
20	Owner rejects a claim	Item stays open; the claim is marked REJECTED	Pass
21	Owner marks an item resolved by hand	Status RESOLVED; shown under the Resolved filter	Pass
22	Owner deletes an item without claims	Item removed; old URL shows “Page not found”	Pass
23	Owner deletes an item that has a claim	Claim and item removed together; claimant’s page still loads	Pass
24	Logout, then press Back	Login required again; private page not shown	Pass
25	Unknown or non-numeric item id, unknown URL	Friendly 404 page	Pass

Table 4: Security measures and how each was checked.

Threat	What the application does	Checked by
Access without login	`AuthFilter` covers every page except login, sign-up and the stylesheet; POST requests from a logged out client are redirected	Browser test
Acting on another user's data	The owner is checked on the server for resolve, delete, approve and reject; anyone else gets HTTP 403	Browser test
Script injection (XSS)	All user text is printed with `<c:out>`; HTML or `<script>` typed into a description or message appears as plain text	Browser test
SQL injection	Only `PreparedStatement` with `?` placeholders	Code review
Stolen passwords	Salted SHA-256 hash; the hash is not kept in the session; the same error text for an unknown user and a wrong password	Code review
Session fixation	Session id changed after login; HttpOnly cookie; 30 minute time-out	Code review
Old private pages	No-cache headers; logout only by POST	Browser test
Opening pages directly	The JSPs are under `WEB-INF`, so they can only be reached through a servlet	Code review
Open redirect after login	Only paths that start with a single slash are accepted	Code review

Table 5: Faults found during testing and how they were fixed.

Fault	Cause	Fix
The server printed its start banner but the web application had not started	A class loading problem in embedded Tomcat while reading `web.xml`	Parent class loader set explicitly; start-up now checks the state of the application and stops with a clear message if it failed
Browse and item pages opened without a login	Their URLs were missing from the filter's list	All pages except login, sign-up and CSS added to `AuthFilter`
Emoji on item cards looked garbled	The included fragment had no encoding and was read as ISO-8859-1	UTF-8 declared for all JSP files in `web.xml`
After login the user could land on "Page not found"	Only the servlet path was remembered, not the full address	Full path and query string are saved
Back button showed a private page after logout	The browser had cached the page	Cache-control headers added in `EncodingFilter`
A raw server error page when `/manage` was typed in the address bar	The servlet handled POST only	GET now redirects to the item or Browse page

4.1 How the objectives were met

The first objective, a complete application with a front end and a back end and no large frameworks, is met by the eleven servlets, the JSP pages and the three DAO classes, which together are small enough for one person to read in an afternoon. Registration and login, with server side checks, salted hashes and sessions, cover the second objective, and posting, search and filtering cover the third and fourth. The claim process (fifth objective) is the heart of the project: a user sends a message, the owner sees it with the sender's details, and only an approval makes the owner's e-mail and phone number visible, which satisfies the sixth objective as well. The seventh objective is handled by the login filter and by the owner check inside every servlet that changes data. The eighth is met by the two transactions, one in the claim decision and one in the delete, and the ninth by the two filters and two listeners. Finally, because the embedded Tomcat and H2 database start inside the program, the tenth objective is met: a user only needs Java and Maven.

4.2 What we learned

The project made the topics of the subject concrete. Writing the servlets showed us how a request really moves through a Java web application, and why forward and redirect are different things: a forward stays inside the server and is used to show a page, while a redirect after a POST stops a refresh from repeating the action. Working with JSP and JSTL taught us to keep Java code out of the pages and to escape everything the user types, which is what stops script injection. With JDBC we learned why prepared statements are necessary and what a transaction is for, because the approve and delete operations really do leave wrong data behind if they are stopped half way. Sessions, filters and listeners showed us that much of the security of a web application is not in any single page but in the pieces that run around all of them.

The most useful lesson came from testing. Opening the finished application in a browser and trying it as a real user found faults that we had not seen while writing the code: the start-up problem in the embedded server, the pages that could be opened without a login, the garbled symbols, the missing return page after login, and the Back button showing a private page after logout. Each one was easy to fix once we saw it, but none was visible in the source. We also learned that a project is not finished when it works on the author's computer. Writing a step by step guide for another person, on another operating system, showed how many things we had taken for granted.

4.3 Future scope

The portal can be extended in several useful directions. Photographs of the objects would make recognising an item much easier, and they need size and type checks and safe storage on the server. The owner and the claimant could be told by e-mail or SMS when a claim arrives or is decided, instead of having to open the dashboard. An administrator role could remove inappropriate reports and handle disputes. A matching feature could compare each new Lost report with the existing Found reports of the same category and place and suggest likely pairs, which would let the portal start the conversation instead of waiting for it. For deployment, the embedded database can be replaced with MySQL or PostgreSQL by changing the properties file, the password hashing can be moved to bcrypt or PBKDF2, anti-CSRF tokens can be added to the forms, and a REST interface to the same back end would allow a mobile application.